Let's demonstrate that by updating the previous example by making `countAndIterate()` also accept an `array` as a valid argument:

```
declare(strict_types=1);

function countAndIterate((ArrayAccess&Countable)|array $data)
{
    print "Data contains " . count($data) . " things:" . PHP_EOL;

    foreach ($data as $datum) {
        var_dump($datum);
    }
}
$niceThings = [
    'a car',
    'a guitar',
    'and a nice cold pint',
];

countAndIterate(new ArrayObject($niceThings));
countAndIterate($niceThings);
```

The output of each call to `countAndIterate()` is identical; notice the type definition for `$data` is now the DNF expression `(ArrayAccess&Countable)|array`.

Standalone Pseudo-types

As mentioned earlier in the section, there are standalone types that do not exist as primitive types but can be used in type declaration.

Literal Types

If a function returns (or accepts) boolean data, you can of course declare the type with the keyword `bool`. For example, allow me to present my patented decision-making algorithm: